

Challenge Técnico – Backend Node.js + TypeScript

Objetivo

El objetivo de este challenge es evaluar conocimientos de desarrollo backend utilizando Node.js, TypeScript, MongoDB, Redis y procesamiento asíncrono mediante colas de mensajes.

La idea no es construir una aplicación completa, sino demostrar buenas prácticas de diseño, arquitectura y calidad de código.

Tiempo total del desarrollo estimado: menos de 3 horas.

Descripción

Desarrollar un sistema de Tiny URL que permita generar URLs cortas, resolverlas y registrar sus accesos.

El frontend puede ser extremadamente simple (HTML básico, React, Vue, EJS, etc.). No será evaluado, únicamente debe permitir probar la funcionalidad.

Funcionalidades requeridas

1. Crear una URL corta

Debe existir una pantalla o endpoint que permita crear una TinyURL a partir de una URL original.

Datos mínimos:

- URL original (obligatoria)
- Alias personalizado (opcional)

Ejemplo:

`https://www.google.com/search?q=nodejs`

Respuesta esperada:

`http://localhost:3000/AbC123`

o

`http://localhost:3000/mi-alias`

2. Resolver una TinyURL

Al acceder a:

`GET /AbC123`

El sistema debe:

- Buscar la URL original.
- Redireccionar al usuario.
- Registrar un evento de consumo de forma asincrónica.

3. Uso de KVS (Redis o similar)

La resolución de una TinyURL debe utilizar Redis como caché.

Se espera un flujo similar a:

Redis ↓ MongoDB (si no existe en cache) ↓ Guardar nuevamente en Redis

Puede utilizar el TTL que considere adecuado.

4. Registro de eventos

Cada acceso a una TinyURL debe generar un evento que sea enviado a una cola de mensajes.

El evento debería contener, como mínimo:

- código de la TinyURL
- fecha
- IP (si está disponible)
- User-Agent (si está disponible)

Lo importante es que el procesamiento sea asíncrono.

5. Persistencia de eventos

Un worker deberá consumir los eventos de la cola y almacenarlos en MongoDB.

6. Endpoint de estadísticas

Debe existir un endpoint similar a:

GET /api/stats/:code

que devuelva información básica:

```
{  "code": "AbC123",  "totalClicks": 125,  "lastClick": "2026-06-10T18:20:15Z" }
```

No es necesario realizar agregaciones complejas.-

Requisitos técnicos

- Node.js
- TypeScript
- MongoDB / MySQL
- Redis
- Cola de mensajes (No hay requisito en cual usar, explicar el porque de la decisión)
- Código organizado y mantenible

Frameworks permitidos:

- Express
- Fastify
- NestJS
- Otros similares

ORM/ODM libre (Mongoose, Prisma, etc.).

Se valorará especialmente

- Buena organización del proyecto.
- Separación en capas (Controllers, Services, Repositories, etc.).
- Correcto uso de TypeScript.
- Manejo de errores.
- Código limpio y legible.
- Validaciones.
- Docker Compose para levantar el entorno.
- Tests unitarios o de integración (opcional).

No será evaluado

- Diseño del frontend.
- CSS.
- Animaciones.
- Estética visual.

El frontend solamente debe permitir probar fácilmente la creación de TinyURLs.

Entrega

Enviar:

- Repositorio Git o ZIP del proyecto.
- Instrucciones para ejecutarlo.
- README con dependencias y pasos de instalación.
- Breve explicación de las decisiones técnicas tomadas.

Aclaración importante

Se permite utilizar herramientas de Inteligencia Artificial (ChatGPT, Claude, Copilot, etc.). Sin embargo, durante la entrevista técnica se solicitará explicar la arquitectura implementada, las decisiones tomadas y el funcionamiento del código presentado. La capacidad de comprender y defender la solución tendrá el mismo peso que la implementación en sí.